

Product Requirements Document

PRD B: Python TCP Chat Application

With Real-Time Web Interface & Network Debugging

Project	LAN TCP Chat System
Document Version	1.0
Created	February 2026
Target Audience	Computer Networks Students
Tech Stack	Python 3.10+, HTML5, CSS3, JavaScript, WebSocket Bridge

Table of Contents

1. Executive Summary
2. Project Overview
3. Learning Objectives
4. System Architecture
5. Functional Requirements
6. Technical Specifications
7. User Interface Design
8. Network Protocol Details
9. Implementation Guidelines
10. Testing & Validation
11. API Documentation
12. Deployment Instructions

1. Executive Summary

This PRD defines the requirements for building a Python-based TCP chat application designed to teach fundamental networking concepts to Computer Science students. The project demonstrates the connection-oriented nature of TCP protocol through a practical, hands-on implementation.

1.1 Project Purpose

Create a lightweight, educational chat system that exposes the inner workings of TCP connections, allowing students to observe and understand:

- Three-way handshake establishment
- Persistent connection maintenance
- Reliable, ordered message delivery
- Socket programming fundamentals
- Client-server architecture patterns

1.2 Key Features

- Multi-client TCP server supporting concurrent connections
- Real-time web interface with network debugging panel
- IP-based pseudonym generation (no authentication required)
- Live network metrics display (IP addresses, ports, byte counts, timestamps)
- Visual connection state monitoring
- Zero database dependency

1.3 Success Criteria

- Server handles at least 10 concurrent clients
- Messages delivered within 100ms on local network
- Web interface displays all network metadata accurately
- System runs reliably for 2+ hours without crashes

- Students can explain TCP vs UDP differences after using the app

2. Project Overview

2.1 Problem Statement

Students learning computer networks often struggle to visualize abstract protocol concepts. Traditional textbook explanations of TCP connections lack the hands-on experience needed for deep understanding. This project bridges the gap between theory and practice by providing a working system that exposes TCP internals.

2.2 Target Users

- Undergraduate Computer Science students in networking courses
- Self-learners exploring socket programming
- Instructors demonstrating TCP concepts in lab environments

2.3 Educational Context

This project directly addresses the following networking concepts from CS424:

- Transport Layer protocols (TCP vs UDP comparison)
- Socket API and system calls
- Port numbers and addressing
- Connection establishment and termination
- Packet structure and headers
- Network byte order and data serialization

2.4 Scope & Constraints

In Scope:

- LAN-only communication (same network segment)
- Text-based messaging
- Python TCP server and client implementations
- WebSocket bridge for browser connectivity
- Real-time network debugging interface

Out of Scope:

- User authentication or registration
- Message persistence (database storage)
- Encryption or security features
- File transfer capabilities
- Mobile native applications
- Internet-wide deployment (NAT traversal, firewall rules)

3. Learning Objectives

3.1 Core Networking Concepts

Students will be able to:

- Explain the TCP three-way handshake process
- Distinguish between connection-oriented and connectionless protocols
- Identify the components of a TCP/IP address (IP + port)
- Describe how TCP ensures reliable data delivery
- Analyze network traffic at the socket level

3.2 Programming Skills

- Implement socket programming in Python using the socket library
- Handle concurrent connections with threading or async/await
- Design client-server architectures
- Build web interfaces that interact with backend services
- Debug network applications using protocol inspection

3.3 System Design

- Architect multi-component distributed systems
- Design RESTful or WebSocket-based APIs
- Implement event-driven programming patterns
- Handle edge cases (disconnections, timeouts, errors)

4. System Architecture

4.1 High-Level Components

The system consists of three main components:

1. TCP Chat Server (Python)

- Listens on a designated port (e.g., 5000)
- Accepts incoming TCP connections from clients
- Maintains list of active connections
- Broadcasts messages to all connected clients
- Tracks network metadata for each connection

2. WebSocket Bridge Server (Python)

- Listens on WebSocket port (e.g., 8080)
- Bridges browser clients to TCP chat server
- Translates WebSocket frames to TCP messages
- Provides JSON-based protocol for metadata exchange

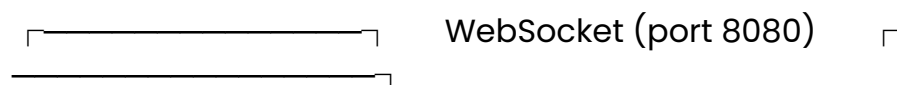
3. Web Interface (HTML/CSS/JavaScript)

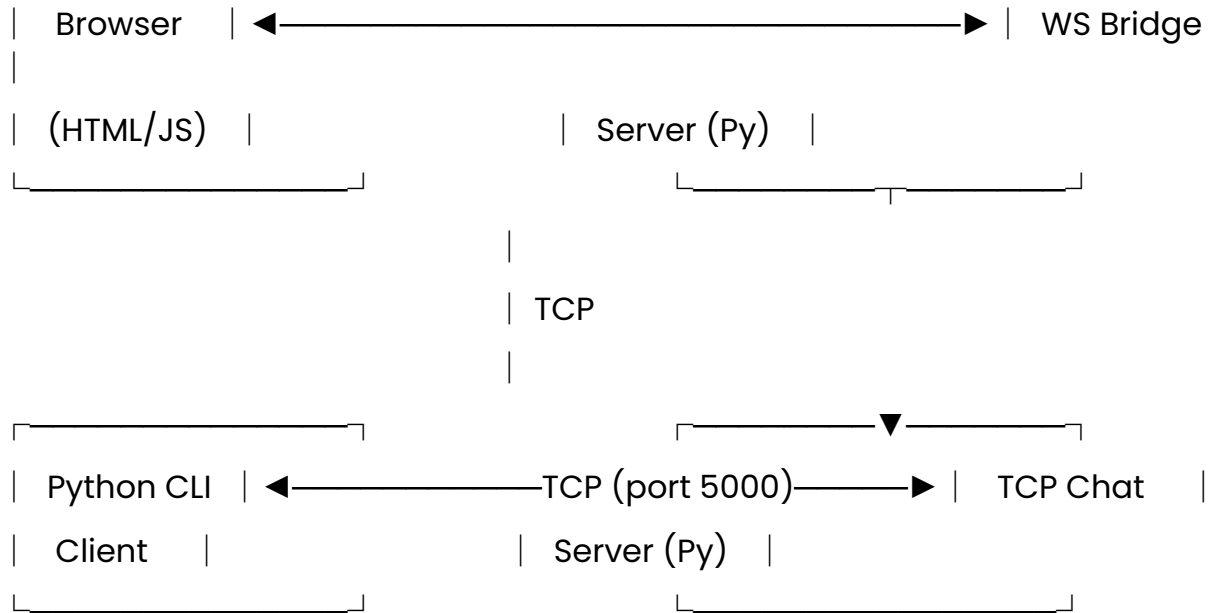
- Single-page application served via HTTP
- Connects to WebSocket bridge
- Displays chat messages and network debugging info
- Shows live connection statistics

4.2 Architecture Diagram

The following diagram illustrates the system architecture:

[Diagram: System Architecture]





4.3 Data Flow

Message transmission flow:

- User types message in web interface
- JavaScript sends WebSocket frame to bridge server
- Bridge extracts message and forwards via TCP to chat server
- Chat server broadcasts to all connected clients (TCP + WS)
- Each client receives message with metadata
- Web interface displays message + network info in debug panel

4.4 Component Interaction Sequence

Connection establishment:

1. User opens web page → JavaScript initiates WebSocket connection
2. WS Bridge accepts connection → Creates TCP socket to chat server
3. Chat server accepts TCP connection → Sends welcome message
4. Server generates IP-based pseudonym (e.g., 'User_192_168_1_42')
5. Client receives pseudonym → Web interface updates user list

6. Connection ready for bidirectional messaging

5. Functional Requirements

5.1 Core Messaging Features

FR-1: Message Broadcasting

- The server MUST broadcast every received message to all connected clients
- Messages MUST be delivered in the order sent (FIFO)
- Each message MUST include sender pseudonym and timestamp

FR-2: Pseudonym Generation

- Server MUST automatically generate pseudonym from client IP address
- Format: User_[IP_with_underscores] (e.g., User_192_168_1_42)
- Pseudonyms MUST be unique per IP address

FR-3: Connection Management

- Server MUST detect client disconnections within 5 seconds
- Server MUST broadcast disconnect notifications to remaining clients
- Server MUST handle graceful shutdowns (SIGINT/SIGTERM)

5.2 Network Debugging Interface

FR-4: Real-Time Network Metrics

- Web interface MUST display for each message:
 - Source IP address and port number
 - Destination IP address and port number
 - Message size in bytes
 - Precise timestamp (millisecond accuracy)
 - Current TCP connection state

FR-5: Connection Statistics Panel

- Interface MUST show:
 - Total connected clients count

- Active connection list with IP addresses
- Total bytes sent/received per connection
- Connection uptime
- Current socket state (ESTABLISHED, TIME_WAIT, etc.)

FR-6: Message History

- Web interface MUST store last 100 messages in memory
- History MUST persist during page refresh (localStorage)
- User MUST be able to clear history manually

6. Technical Specifications

6.1 Server Requirements

Component	TCP Chat Server
Language	Python 3.10+
Core Libraries	socket, threading, json, datetime
Port	5000 (configurable)
Protocol	TCP (stream socket)
Concurrency	Threading (one thread per client)
Max Clients	50 (configurable)
Message Format	JSON-encoded strings with newline delimiter

Component	WebSocket Bridge Server
Language	Python 3.10+
Core Libraries	websockets, asyncio, socket, json
Port	8080 (configurable)
Protocol	WebSocket (RFC 6455)
Concurrency	asyncio event loop
Bridge Mode	Bidirectional proxy between WS and TCP

6.2 Client Requirements

Python CLI Client (Optional):

Language	Python 3.10+
Libraries	socket, threading, sys
Usage	python client.py <server_ip> <port>

Web Client:

Technologies	HTML5, CSS3, JavaScript (ES6+)
Dependencies	None (vanilla JavaScript)
Browser Support	Chrome 90+, Firefox 88+, Safari 14+
WebSocket API	Native browser WebSocket

6.3 Message Protocol

All messages exchanged between components use JSON format:

Message Structure:

```
{
  "type": "chat",
  "username": "User_192_168_1_42",
  "message": "Hello",
  "timestamp": "2026-02-25T14:30:00.123Z",
  "metadata": {
    "source_ip": "192.168.1.42",
    "source_port": 52341,
    "dest_ip": "192.168.1.1",
    "dest_port": 5000,
    "bytes": 156,
    "connection_state": "ESTABLISHED"
  }
}
```

System Message Structure:

```
{
  "type": "system",
  "message": "User_192_168_1_42 joined",
  "timestamp": "2026-02-25T14:30:00.123Z"
}
```

9. Implementation Guidelines

9.1 Development Phases

Phase 1: TCP Chat Server (Week 1)

- Implement basic TCP server with `socket.socket()`
- Accept single client connection
- Echo messages back to sender
- Add threading for multiple clients
- Implement broadcast functionality
- Add pseudonym generation from IP address

Phase 2: Python CLI Client (Week 1)

- Create basic TCP client
- Implement send/receive in separate threads
- Add command-line interface
- Test with multiple client instances

Phase 3: WebSocket Bridge (Week 2)

- Set up WebSocket server with `websockets` library
- Create TCP client socket to connect to chat server
- Implement bidirectional message forwarding
- Add JSON protocol for metadata

Phase 4: Web Interface (Week 2)

- Build HTML structure with three-column layout
- Style with CSS (dark theme)
- Implement WebSocket connection in JavaScript
- Add message display and input handling
- Create network debug panel

Phase 5: Integration & Testing (Week 3)

- Test with multiple browsers and CLI clients simultaneously
- Verify network metadata accuracy
- Fix bugs and edge cases
- Add error handling and reconnection logic

Document Control

Version	1.0
Date	February 25, 2026
Author	Computer Networks Course
Status	Final
Next Review	March 2026